

CSE 433 – Embedded Systems

Project 1 Report

Sümeyye Zeynep Gürbüz: 200104004039

Talha Kaymak: 200104004052

FPGA-Based Edge Detection using the Sobel Operator

Project Overview

This project implements a Sobel-based edge detection system on an FPGA. The system reads a grayscale image, computes edges using Sobel kernels, and outputs a binarized edge image. Initially written in C, the algorithm is later mapped to a hardware design consisting of a datapath, a control unit (FSM), and memory components.

Part 1: C Software Reference Design

Functionality:

- The C code reads a grayscale PGM image.
- Applies Sobel edge detection using two convolution kernels G_x and G_y .
- Computes the edge magnitude at each pixel using:
$$\text{magnitude} = |\text{sumX}| + |\text{sumY}|$$
- If the magnitude exceeds a threshold (185), it marks the pixel as edge (255), else background (0).
- Writes the resulting edge matrix to a new PGM image.

Algorithm:

```
sumX += img[y + i][x + j] * Gx[1 + i][1 + j];  
sumY += img[y + i][x + j] * Gy[1 + i][1 + j];
```

This operation is performed over a 3×3 window centered at each pixel (excluding borders).

Part 2: Hardware Mapping

Key Hardware Components:

Component	Description
<i>Datapath</i>	Performs all arithmetic: convolution, accumulation, comparison
<i>Control Unit (FSM)</i>	Controls operation flow: scan pixels, manage loops, reset accumulators
<i>Image ROM</i>	Stores the input image, initialized via .mif file
<i>Edge RAM</i>	Stores the computed edge result
<i>Gx/Gy Kernels</i>	Hardcoded as 2D registers inside datapath

Part 3: Datapath Design

The datapath is the arithmetic and storage engine of the edge detection system. It performs convolution, absolute value summation, threshold comparison, and controls image memory addressing. It is highly parameterized and controlled externally by the FSM.

Registers & Variables:

Register	Purpose
<i>x, y</i>	Current pixel coordinates
<i>i, j</i>	Kernel window offsets (range: -1 to 1)
<i>sumx, sumy</i>	Accumulated horizontal and vertical gradients
<i>magnit</i>	Edge strength: $\text{abs}(\text{sumx}) + \text{abs}(\text{sumy})$
<i>imgAddr</i>	Computed image address from $(x+i, y+j)$
<i>gxAddr, gyAddr</i>	Flattened index for accessing Gx/Gy kernels
<i>edgWrAddr</i>	Address to write the result edge pixel

edgData Pixel value to write to edge image RAM (0 or 255)

Arithmetic Units:

- Multiplication of image pixel (`imgDout`) by `gxDout` and `gyDout`
- Accumulation of the results into `sumx`, `sumy`
- Optional negation (used to compute `abs()` by multiplying by -1)
- Threshold logic against parameter `TH` (185 by default)

Control Inputs:

Signal	Function
<code>ldX</code> , <code>ldY</code> , <code>ldI</code> , <code>ldJ</code>	Load and update image and kernel loop counters
<code>ldSumx</code> , <code>ldSumy</code>	Clear or accumulate <code>sumx</code> / <code>sumy</code>
<code>Ssumxop</code> , <code>Ssumyop</code>	Select between accumulation or negation
<code>ldMagnit</code>	Triggers calculation of <code>magnit</code>
<code>Scompl</code> , <code>Scompr</code>	Encodes condition checks for FSM (e.g., <code>x < WIDTH-1</code>)
<code>Sedg</code>	Chooses between writing 0 or 255 to output RAM

External Memory Interfaces:

- `img_rom`: Image read-only memory
- `Gx`, `Gy`: ROMs storing 3×3 Sobel kernels
- `edg_ram_dual`: Dual-port RAM to store the final edge-detected output
- All read/write memory operations are addressed using flattened: $(x + j) + \text{WIDTH} * (y + i)$ form.

Input/Output:

- **Input ports:** `imgAddr`, `gxAddr`, `gyAddr`, and control signals from FSM
- **Output ports:** `edgeDout` (pixel result)

The design is modular and uses only signed arithmetic for Sobel computation. All computations are gated by control signals, meaning the datapath is fully decoupled from control logic, enabling clean pipelining and reuse.

Part 4: Control Unit Design

The `control_unit` is a finite state machine (FSM) that orchestrates all actions in the datapath by enabling the appropriate control signals. It fully implements the `for-y → for-x → for-i → for-j` structure of the Sobel algorithm.

FSM Behavior

Key States and Flow:

State	Role
S00	Reset and initialize x and y to 0
S01	Begin first y loop (outer loop)
S02	Check $y < HEIGHT - 1$; increment x inside image bounds
S03-S05	Prepare for kernel iteration; reset i, j, and accumulators
S06-S08	Load kernel values, image pixels, and perform accumulation
S09-S12	Conditional negation for sumx, sumy to compute abs()
S13-S15	Compute magnitude and compare to threshold
S16-S18	If $magnitude > THRESHOLD$, write 255, else write 0

The FSM uses signals like `Scomp1` and `Scompr` to evaluate loop and comparison conditions ($i \leq 1$, $x < WIDTH$, $sumx < 0$, etc.) using the `lt` and `lteq` feedback signals from the datapath.

Control Signals

Signal	Function
<code>ldX, ldY, ldI, ldJ</code>	Control loop counter loads/increments
<code>ldSumx, ldSumy</code>	Enable resetting or accumulating gradient sums
<code>Ssumxop, Ssumyop</code>	Select operation: accumulation or negation
<code>ldMagnit</code>	Compute magnitude
<code>we, Sedg</code>	Enable result writing to RAM
<code>ldImgAddr, ldGxAddr, ldGyAddr</code>	Generate memory addresses for reads
<code>Scomp1, Scompr</code>	Parameterized comparisons ($x < N$, $mag < TH$)

All control signals are driven combinatorially in `always @(*)`, while the FSM state itself updates on the positive clock edge.

Design Challenge: Representing a 256×256 Image in FPGA

The Problem:

Representing a 256×256 grayscale image (with 8-bit pixel depth) in FPGA poses challenges due to:

- **Memory constraints:** A 256×256 image = 65,536 pixels = **64 KB** of data.
- **Data loading:** The image needs to be transferred into the FPGA or preloaded at synthesis.
- **Efficient access:** Each pixel must be accessible by address during convolution.

Solution: Dual Strategy with IP RAM + UART

1. On-Chip RAM (via IP Catalog)

- Used **Quartus IP Catalog** to generate a single-port or dual-port RAM block.
- Initialized the RAM with a `.mif` (Memory Initialization File), generated from a PGM image.
- This allows the image to be stored in ROM at synthesis time, with pixel access via computed addresses like:
$$\text{imgAddr} = (y+i) \times 256 + (x+j)$$
- Multiple `.mif` files = multiple ROMs → selected via FPGA switches.

2. UART (for runtime flexibility)

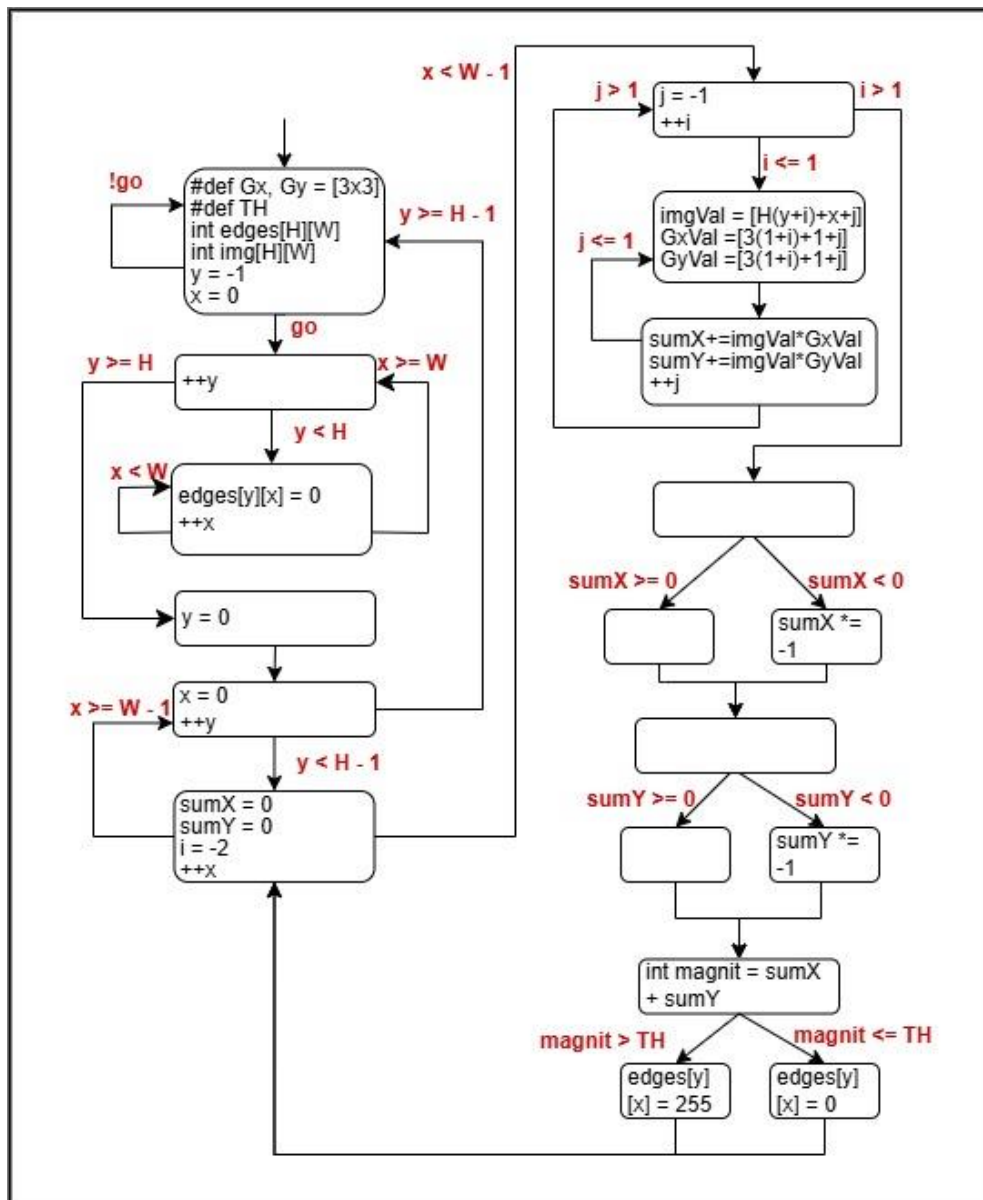
- To support dynamic image loading or result retrieval, a **UART interface** is integrated.
- Enables transferring image data to/from FPGA over serial.
- Particularly useful when `.mif`-based ROMs become limiting for image variety or debugging.

Summary:

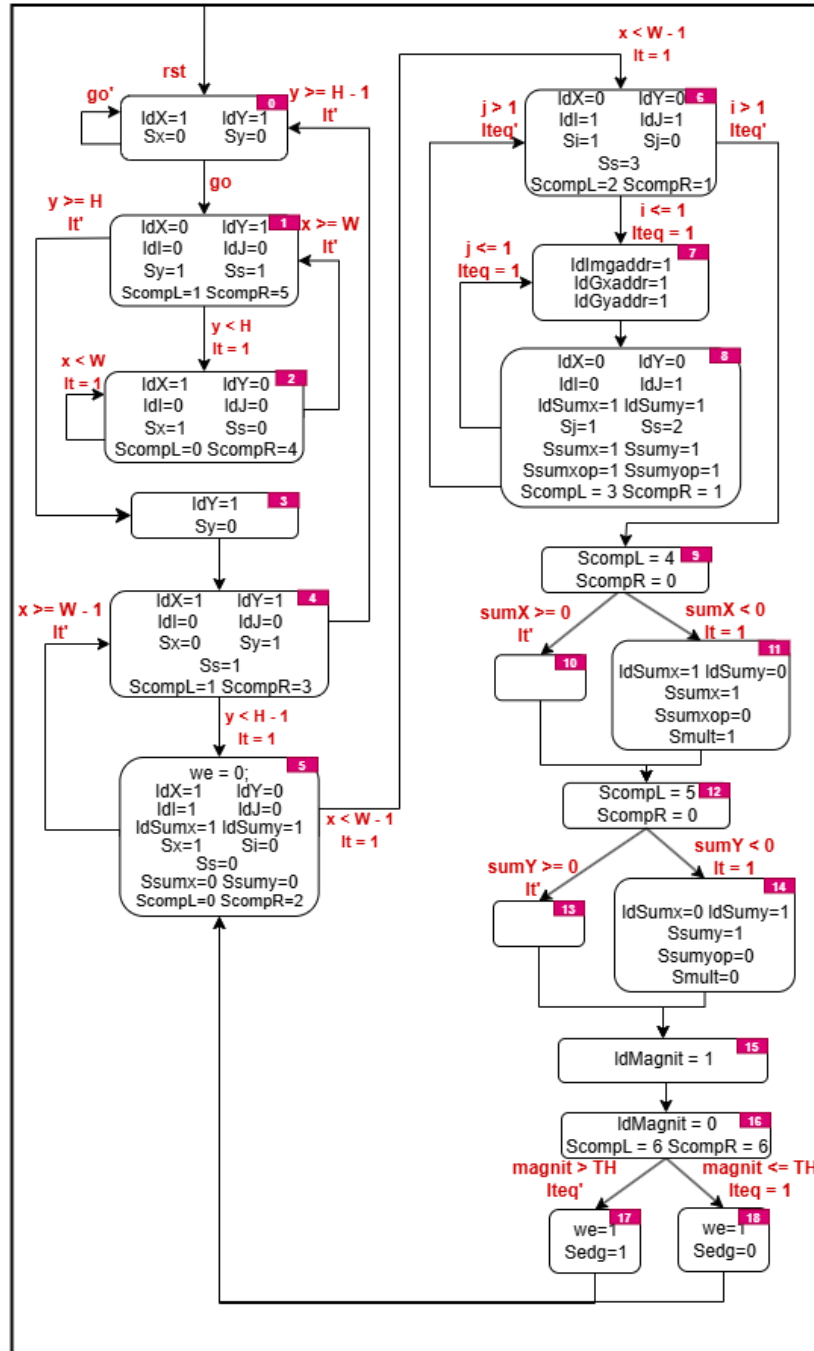
- The FSM reflects the structure of the C code's nested loops.

- Each FSM state performs **one atomic operation**, promoting clarity and pipelining.
- The FSM is well-optimized, with only 19 states managing all Sobel kernel operations over the image space.
- Condition flags (1t, 1teq) help drive loop control decisions efficiently.

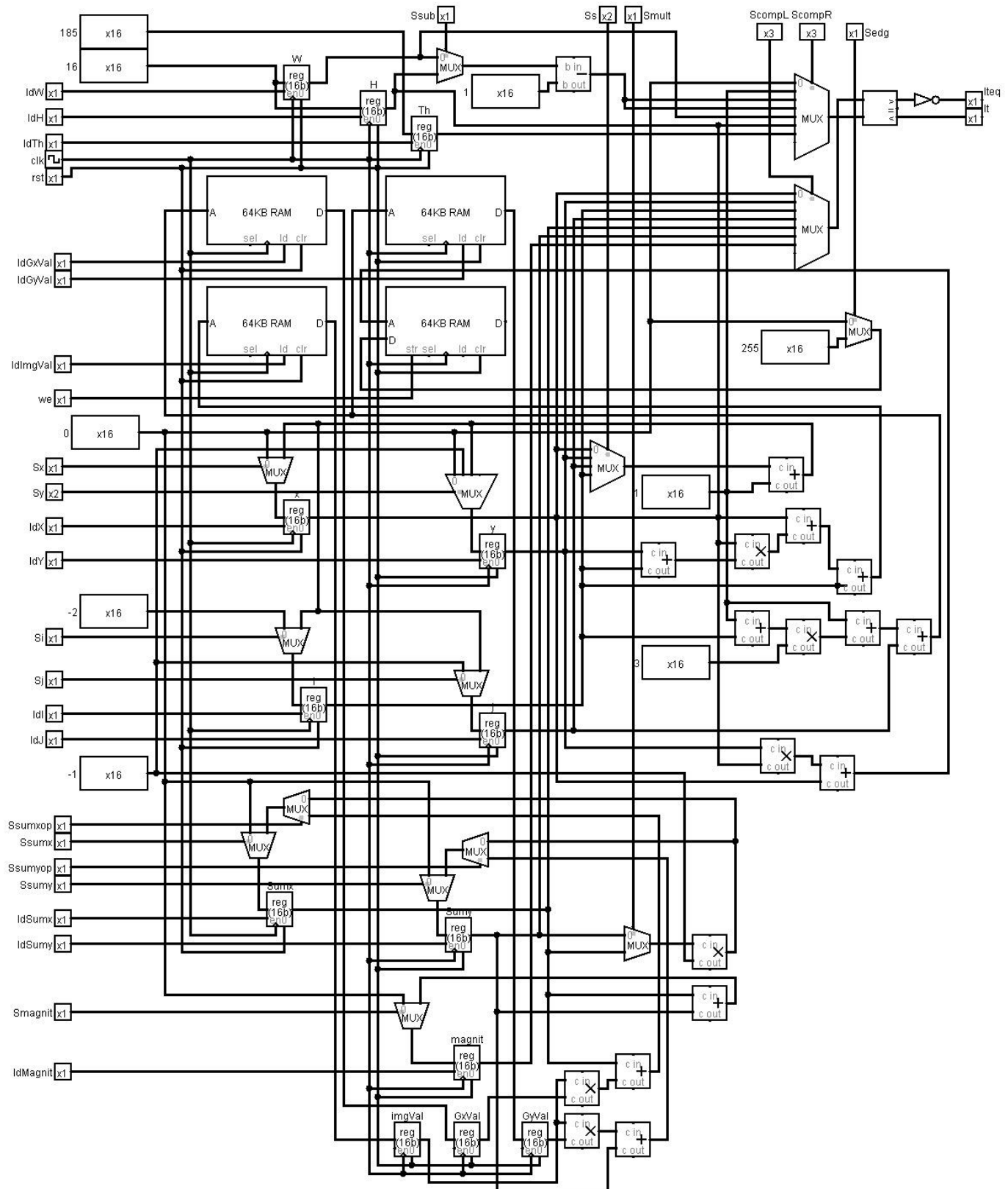
FSMD & Datapath Block Diagrams



FSM Diagram



FSMD with Signals



Datapath Circuit Design